

HOW THE SYSTEM WORKS

A short guide to how our online screening and booking system is built, what each part does, and how we were able to implement the key features.

Laravel 12 · PHP 8.2

MySQL

Blade templates

Plain CSS + JavaScript

PayMongo

PHPUnit · 87 tests

1 What the system is

The system has two sides that share one database:

Customer site · /cinemathequecentredavao

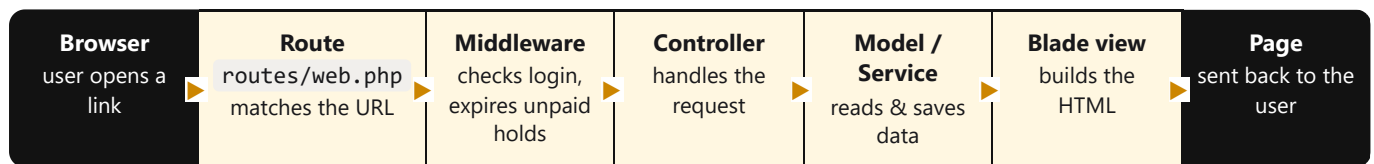
For moviegoers. No account needed. They browse screenings, view film details, pick seats, fill in who's coming, pay online (paid screenings), and look up their booking and e-ticket.

Staff area · /ccdadmin

For Cinematheque staff only (login required). They manage films and screenings, approve or cancel bookings, check in attendees at the door, view reports, and manage staff accounts.

2 How a page is made (the request flow)

Laravel follows the **MVC** pattern (Model–View–Controller). Every page the user opens goes through the same path:



Where things are in the code

FOLDER / FILE	WHAT IT DOES
routes/web.php	Lists every URL of the site and which controller handles it.
app/Http/Controllers	The logic for each page: <code>PublicScreeningController</code> (listing & film page), <code>BookingController</code> (seats, details, payment), and <code>Staff/...</code> for the admin pages.
app/Models	One class per database table (Movie, Screening, Seat, Reservation, Payment...) and how they relate.
app/Services	Bigger jobs kept out of controllers: PayMongo payments, sending emails, expiring unpaid bookings, screening tags.
app/Http/Requests	Form validation rules, e.g. one attendee per seat and Philippine mobile numbers (+63 9XX XXX XXXX).
app/Policies	Who may do what in the staff area (e.g. a cancelled booking can't be approved).
resources/views	Blade templates (HTML pages). <code>layouts/app</code> holds the shared header and footer.

<code>public/css/cinematheque.css</code> , <code>public/js/cinematheque.js</code>	All customer-site styling and the small amount of JavaScript (tabs, carousel, trailer player, seat picker).
<code>database/migrations</code> , <code>database/seeder</code> s	Build the tables step by step, and fill them with demo data (staff accounts, 120 seats, films, screenings).
<code>tests/Feature</code>	87 automated tests that check the main features still work after every change.

3 The database

The data is stored in MySQL. The main groups of tables are:

GROUP	TABLES	WHAT THEY HOLD
Films	movies, actors, directors, genres (+ link tables)	Title, year, runtime, rating, synopsis, poster, curator's note, trailer link, cast and genres.
Venue	seats	The 120 fixed seats of the hall: rows A–J, seats 1–12.
Screenings	screenings	Each showing: film, date, start/end time, free or paid, price, capacity.
Bookings	reservations, reservation_seats, reservation_attendees	A booking (with its booking reference and status), the seats in it, and one attendee's details per seat.
Payments & door	payments, attendances	Online payment status for paid screenings, and who was checked in at the door.
Staff	users	Staff accounts (AVT / PDO), active or deactivated.

4 The customer booking flow

- 1 Browse screenings.** The home page shows a looping spotlight carousel of featured films and poster cards in four tabs: *Now Showing* (next 7 days), *Advance Booking* (within 30 days), *Coming Soon*, and *Special Screenings*. Screenings of the same film are grouped into one card.
- 2 Open a film.** The film page shows the poster, details "at a glance", the curator's note, the trailer (if any), and every showtime as a ticket with a seat-availability dot (green, orange or red).
- 3 Pick seats.** The seat map shows the hall as it is (screen, 10 rows × 12 seats, exits, entrance). Taken seats are disabled; up to 10 seats per booking. The first seat picked is the main booker.
- 4 Who's coming?** One short form per seat (name, age, sex, school/company, mobile, email, optional senior/PWD ID). The summary card on the right shows the poster, seats and total.
- 5 Submit.** The booking is saved as *pending*. **Free** screenings wait for staff approval. **Paid** screenings go to PayMongo's checkout, and the seats are held for 15 minutes.
- 6 Confirmation.** Once approved (free) or paid (paid), the booking becomes *confirmed* and the e-ticket is emailed. Customers can also open it anytime under *Find my booking* with their booking reference or email.

5 How we implemented the key features

No double-booking of seats. When a booking is saved, the code locks that screening's row in the database (`lockForUpdate`) inside a transaction, so two people booking at the same moment are handled one after the other. As a second safety net, the database itself has a *unique* rule on (screening, seat), so the same seat can never be saved twice. If it happens, the customer is asked to choose again.

Safe online payments (PayMongo). We never mark a booking as paid just because the customer came back from the payment page. The system asks PayMongo's API directly and only accepts it when PayMongo reports it *paid* for the right booking and amount. A **webhook** also covers customers who pay and close the tab; it is checked with PayMongo's signature (HMAC-SHA256) before it's trusted.

Unpaid holds expire by themselves. A paid booking holds its seats for 15 minutes. A middleware runs on every page visit and cancels overdue unpaid bookings, releasing their seats, so no cron job is required (a scheduled command also exists for servers that use one).

Secure staff area. Staff pages need a login (`auth`), and a second check (`EnsureStaffIsActive`) logs out deactivated accounts right away. Login attempts are rate-limited, and policies stop actions that don't make sense, like approving a cancelled booking.

Emails that never block a booking. `ReservationMailer` sends the pending, approved and cancelled emails. If sending fails, the booking is still saved, the error is logged, and staff can press *Resend*.

Smart screening tags. Tags like *35mm Film Print*, *Director Q&A*, *Restored Classic* and *Curator's Pick* are worked out from data staff already enter (screening title, release year, curator's note), so no extra fields were needed.

Works with or without JavaScript. Pages are built on the server first, so everything works even if JavaScript fails. JavaScript only adds extras: the tab bar, the looping carousel (it clones slides at both ends so a film always peeks in on each side), the trailer pop-up, and the live seat total.

Brand design and themes. All colours are CSS variables in one place, following the FDCP colourway (yellow, black and white). The site opens in light mode; a toggle switches to dark mode and the browser remembers the choice.

6 The staff area

PAGE	WHAT STAFF CAN DO
Dashboard & search	See today's and upcoming screenings and bookings that need action; search bookings, screenings and films from the top bar.
Screenings	Create and edit screenings; each screening page lists its attendees for check-in at the door (one person or a whole group) and can approve all pending free bookings at once.
Reservations	Filter bookings, approve or cancel them, resend emails, and refresh a payment's status from PayMongo.
Reports	Attendance and payment totals for a date range, with CSV export.

Films	Add films with poster upload (drag and drop), director, cast, genres, curator's note and trailer link.
Staff accounts	Add, edit and deactivate staff users.

7 How we built and tested it

- **Step by step with migrations.** Every change to the database (new fields like the curator's note, or relabelling the seats to 10 rows × 12) is a migration file, so every copy of the project can be updated the same way and changes can be rolled back.
- **Demo data with seeders.** One command creates the staff accounts, the 120 seats, the film catalogue and sample screenings and bookings, so the system can be demonstrated right away.
- **Automated tests.** 87 feature tests check the main rules, like seat locking, payment verification, expiry of unpaid bookings, staff access, mobile number validation and the seat layout. We ran them after every change to catch bugs early.
- **Revisions with the client.** The design went through several rounds of client feedback: the screenings page was redesigned with Ayala Malls Cinema as a layout reference, the colours were changed to FDCP's yellow/black/white, the booking pages were reorganised, and the seat map was redrawn to match the actual hall.

8 Running the system

With XAMPP's MySQL running, from the project folder (full steps in `SETUP.md`):

```
composer install
php artisan key:generate
php artisan migrate --seed
php artisan storage:link
php artisan serve
```

Then open `http://127.0.0.1:8000` for the customer site and `/ccdadmin` for the staff area. Run the tests with `php artisan test`.